

IncidentFlow

Guide de Présentation Code & Persistance

Repères Fichier par Fichier pour la Soutenance

**Affichage Frontend (React) & Persistance Backend (Spring Boot /
JPA / PostgreSQL)**

Localisation des Données des Utilisateurs, Rôles & Workflows Dynamiques

Auteur :

NetMar Technologies
Équipe IncidentFlow

Destination :

Démonstration & Soutenance
Encadrant / Professeur

Juillet 2026

Table des matières

1	Discours de Présentation Synthétique (À dire à l'oral)	2
2	Couche Frontend : Affichage et Édition (React UI)	2
2.1	1. Stockage dans l'État React (<code>useState</code>) et Chargement REST	2
2.2	2. Affichage et Modification Dynamique du Rôle par Transition	3
2.3	3. Envoi de la Sauvegarde au Backend	3
3	Couche Backend : Persistance JPA, Services et PostgreSQL	4
3.1	1. Point d'Entrée HTTP (Contrôleur REST)	4
3.2	2. Logique Transactionnelle de Persistance (<code>WorkflowService</code>)	4
3.3	3. Modèle d'Entités JPA (<code>Workflow</code> , <code>WorkflowTransition</code> , <code>User</code> , <code>Role</code>)	5
4	Tableau Récapitulatif des Repères dans le Code	6
5	Schéma Synthétique du Flux de Persistance	6

1 Discours de Présentation Synthétique (À dire à l'oral)

Lorsque le professeur ou l'encadrant vous demande : « Montrez-moi dans le code où les données s'affichent et où elles sont persistées », vous pouvez présenter l'architecture avec la démonstration suivante :

Discours à présenter au Professeur / Encadrant

« Monsieur, l'application est découpée en deux couches principales :

1. **Côté Frontend (React)** : Les données (Workflows, Utilisateurs, Rôles) sont chargées depuis l'API REST via la méthode `fetchWorkflows()` dans `App.jsx`. Elles sont stockées dans des états React (`useState`). L'utilisateur peut modifier en temps réel le rôle autorisé pour chaque transition dans la liste déroulante (`<select>`). Lors du clic sur Enregistrer, la fonction `handleSaveWorkflowGlobally()` transmet le JSON au backend par une requête HTTP PUT.
2. **Côté Backend (Spring Boot & PostgreSQL)** : Le contrôleur REST `WorkflowController` réceptionne la requête sur PUT `/api/workflows/{id}` et la transmet à `WorkflowService`. Le service exécute une transaction JPA (`@Transactional`) qui nettoie les anciennes entités enfant (`orphanRemoval`) et persiste les nouvelles entités `WorkflowState` et `WorkflowTransition` (comprenant la colonne `role_required`) directement dans PostgreSQL via Spring Data JPA. »

2 Couche Frontend : Affichage et Édition (React UI)

2.1 1. Stockage dans l'État React (`useState`) et Chargement REST

Les données du workflow actif et de la liste des utilisateurs et rôles sont gérées dans l'état principal du composant React.

Localisation dans le Code :

Fichier : `frontend/src/App.jsx` (Lignes 196 à 199 & Lignes 825 à 837)

```

1 // Lignes 196-199 : Etats React pour les workflows, utilisateurs et roles
2 const [workflows, setWorkflows] = useState([]);
3 const [activeWorkflow, setActiveWorkflow] = useState(null);
4 const [rolesList, setRolesList] = useState([]);
5
6 // Lignes 825-837 : Chargement depuis le Backend via API REST
7 const fetchWorkflows = async () => {
8   const res = await fetch(`${API_BASE}/workflows`, { headers: getHeaders
9     () });
10   if (res.ok) {
11     const data = await res.json();
12     setWorkflows(data);
13     if (data.length > 0) setActiveWorkflow(data[0]);
14   }
15 };

```

Listing 1 – Stockage local et appel API de recuperation dans App.jsx

2.2 2. Affichage et Modification Dynamique du Rôle par Transition

Chaque transition (`fromState` → `toState`) est affichée avec une liste déroulante interactive permettant d'assigner le rôle autorisé (`roleRequired`).

Localisation dans le Code :

Fichier : `frontend/src/App.jsx` (Lignes 1457 à 1468 & Lignes 4236 à 4280)

```

1 // Lignes 1457-1468 : Fonction de mise a jour du role autorise
2 const handleUpdateTransitionRole = (fromState, toState, roleRequired) =>
3 {
4   if (!activeWorkflow) return;
5   const updatedTrans = activeWorkflow.transitions.map(t => {
6     if (t.fromState === fromState && t.toState === toState) {
7       return { ...t, roleRequired: roleRequired || null };
8     }
9     return t;
10  });
11  const updatedWf = { ...activeWorkflow, transitions: updatedTrans };
12  setActiveWorkflow(updatedWf);
13 };

```

Listing 2 – Mise a jour dynamique de l'état local dans App.jsx

```

1 // Lignes 4236-4280 : Rendu HTML/JSEX de la liste des transitions avec <
  select>
2 {activeWorkflow.transitions.map((t, idx) => (
3   <div key={idx} className="transition-row">
4     <span>{t.fromState} -> {t.toState}</span>
5
6     <!-- Liste deroulante du Role Autorise -->
7     <select
8       value={t.roleRequired || ''}
9       onChange={(e) => handleUpdateTransitionRole(t.fromState, t.toState,
10         e.target.value)}
11     >
12       <option value="">Tous les utilisateurs</option>
13       <option value="Administrateur">Administrateur</option>
14       <option value="Responsable">Responsable</option>
15       <option value="Operateur">Operateur</option>
16     </select>
17   </div>
18 )})

```

Listing 3 – Rendu JSX de la liste deroulante de role par transition dans App.jsx

2.3 3. Envoi de la Sauvegarde au Backend

Localisation dans le Code :

Fichier : `frontend/src/App.jsx` (Lignes 1310 à 1339)

```

1 // Lignes 1310-1339 : Publication globale du workflow vers le Backend
2 const handleSaveWorkflowGlobally = async () => {
3   const res = await fetch(`${API_BASE}/workflows/${activeWorkflow.id}`, {
4     method: 'PUT',
5   });
6 }

```

```

5     headers: getHeaders(),
6     body: JSON.stringify(activeWorkflow)
7 });
8 const updated = await res.json();
9 setActiveWorkflow(updated);
10 };

```

Listing 4 – Envoi HTTP PUT vers l'API Backend dans App.jsx

3 Couche Backend : Persistance JPA, Services et PostgreSQL

3.1 1. Point d'Entrée HTTP (Contrôleur REST)

Localisation dans le Code :

Fichier : backend/src/main/java/com/netmar/incidentflow/controller/WorkflowController.java
(Lignes 40 à 44)

```

1 @PostMapping("/{id}")
2 public Workflow updateWorkflow(@PathVariable Long id, @RequestBody
   Workflow workflowDetails) {
3     workflowDetails.setId(id);
4     return workflowService.saveWorkflow(workflowDetails);
5 }

```

Listing 5 – Endpoint PUT d'enregistrement dans WorkflowController.java

3.2 2. Logique Transactionnelle de Persistance (WorkflowService)

Le service exécute l'enregistrement en base de données avec nettoyage des entités orphelines et gestion des versions.

Localisation dans le Code :

Fichier : backend/src/main/java/com/netmar/incidentflow/service/WorkflowService.java
(Lignes 51 à 160)

```

1 @Transactional
2 @CacheEvict(value = {"workflows", "workflows-category"}, allEntries =
   true)
3 public Workflow saveWorkflow(Workflow workflow) {
4     validateWorkflowGraph(workflow); // Verification DFS de l'integrite
   du graphe
5
6     Workflow existing = workflowRepository.findById(workflow.getId())
   .orElseGet(() -> workflow);
7
8
9     if (existing != workflow) {
10         existing.setName(workflow.getName());
11         existing.setCategory(workflow.getCategory());
12         existing.setActive(workflow.isActive());
13
14         // Supprime les anciens etats et insere les nouveaux (
   orphanRemoval)

```

```

15     existing.getStates().clear();
16     for (WorkflowState state : workflow.getStates()) {
17         existing.getStates().add(WorkflowState.builder()
18             .name(state.getName()).label(state.getLabel())
19             .colorClass(state.getColorClass()).active(state.isActive())
20             .workflow(existing).build());
21     }
22
23     // Supprime les anciennes transitions et insere les nouvelles
24     // avec roleRequired
25     existing.getTransitions().clear();
26     for (WorkflowTransition transition : workflow.getTransitions()) {
27         existing.getTransitions().add(WorkflowTransition.builder()
28             .fromState(transition.getFromState()).toState(
29                 transition.getToState())
30             .roleRequired(transition.getRoleRequired())
31             .requiresComment(transition.isRequiresComment())
32             .workflow(existing).build());
33     }
34     return workflowRepository.save(existing); // Execution SQL via
        Hibernate

```

Listing 6 – Algorithme de persistance transactionnel dans WorkflowService.java

3.3 3. Modèle d'Entités JPA (Workflow, WorkflowTransition, User, Role)

Localisation dans le Code :

Fichiers :

- backend/src/main/java/com/netmar/incidentflow/model/Workflow.java (Lignes 35 à 41)
- backend/src/main/java/com/netmar/incidentflow/model/WorkflowTransition.java (Lignes 26 à 36)
- backend/src/main/java/com/netmar/incidentflow/model/Role.java (Lignes 22 à 31)

```

1 // Dans Workflow.java : Cascade complete et suppression d'orphelins
2 @OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL,
3     orphanRemoval = true, fetch = FetchType.EAGER)
4 private List<WorkflowState> states = new ArrayList<>();
5
6 @OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL,
7     orphanRemoval = true, fetch = FetchType.EAGER)
8 private List<WorkflowTransition> transitions = new ArrayList<>();
9
10 // Dans WorkflowTransition.java : Propriete du role requis
11 @Column(name = "role_required")
12 private String roleRequired;

```

Listing 7 – Annotations de cascade JPA dans Workflow.java et WorkflowTransition.java

4 Tableau Récapitulatif des Repères dans le Code

Composant	Rôle / Fonction	Fichier	Lignes
Formulaire UI	Affichage transitions & rôle	frontend/src/App.jsx	L4236–4280
Mise à jour State	Modification rôle dynamique	frontend/src/App.jsx	L1457–1468
Publication REST	Appel HTTP PUT vers API	frontend/src/App.jsx	L1310–1339
Contrôleur REST	Endpoint PUT /api/workflows	WorkflowController.java	L40–44
Service Persistance	Sauvegarde transactionnelle JPA	WorkflowService.java	L51–160
Entité Workflow	Cascade JPA & Orphan Removal	Workflow.java	L35–41
Entité Transition	Colonne role_required	WorkflowTransition.java	L26–36
Matrice RBAC	Jointure Roles/Permissions	Role.java	L22–31

TABLE 1 – Tableau de repérage rapide pour les questions du professeur.

5 Schéma Synthétique du Flux de Persistance

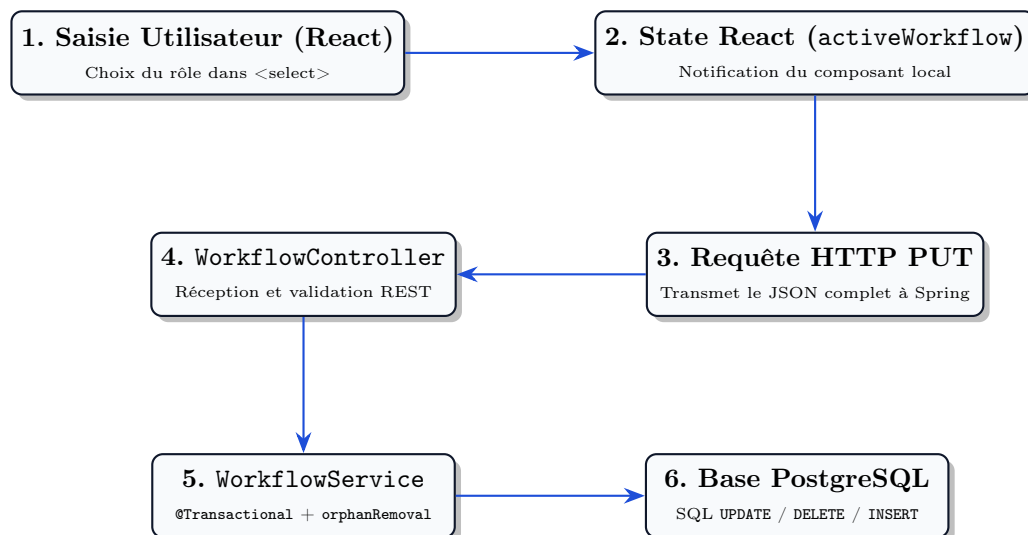


FIGURE 1 – Flux complet d’affichage et de persistance des données.